Public Bus Live Tracking & Crowding Estimator

Lab 3 – Architecture Selection & Written Justification

Architecture Selection

We choose a **Microservices Architecture** for the Public Bus Live Tracking & Crowding Estimator. The system has distinct responsibilities: GPS telemetry ingestion, live bus tracking and ETA calculation, ticketing-event processing, crowding estimation, route and stop management, and commuter/controller interfaces. These responsibilities can be separated into independently managed services that communicate through defined APIs and data interfaces.

Architectural Choice

A microservices approach separates the Live Tracking & ETA service from the Crowding Estimation service and the Bus/Route service. This separation lets high-frequency GPS updates be processed without blocking ticketing events or ordinary route queries. A shared transit database stores current positions, route and stop data, ETA records, ticketing events and estimated crowd levels. The backend API gateway provides a consistent access point for the commuter app and fleet controller portal.

Reason 1 – Independent Scaling and Performance

GPS devices are expected to send updates every five seconds. The tracking and ETA service can therefore be scaled according to telemetry volume, independently of the crowding estimator or user-facing APIs. This helps the system meet the problem statement's target of supporting up to 1,000 active vehicles while keeping arrival information responsive.

Reason 2 – Fault Isolation and Modularity

A failure in ticketing-event processing should not stop commuters from viewing the last known bus position, and a temporary mapping or traffic-provider failure should not disable crowding estimates. Separating these functions allows failures to be detected and handled within the affected service. Stale GPS data can be flagged rather than shown as a fresh live update.

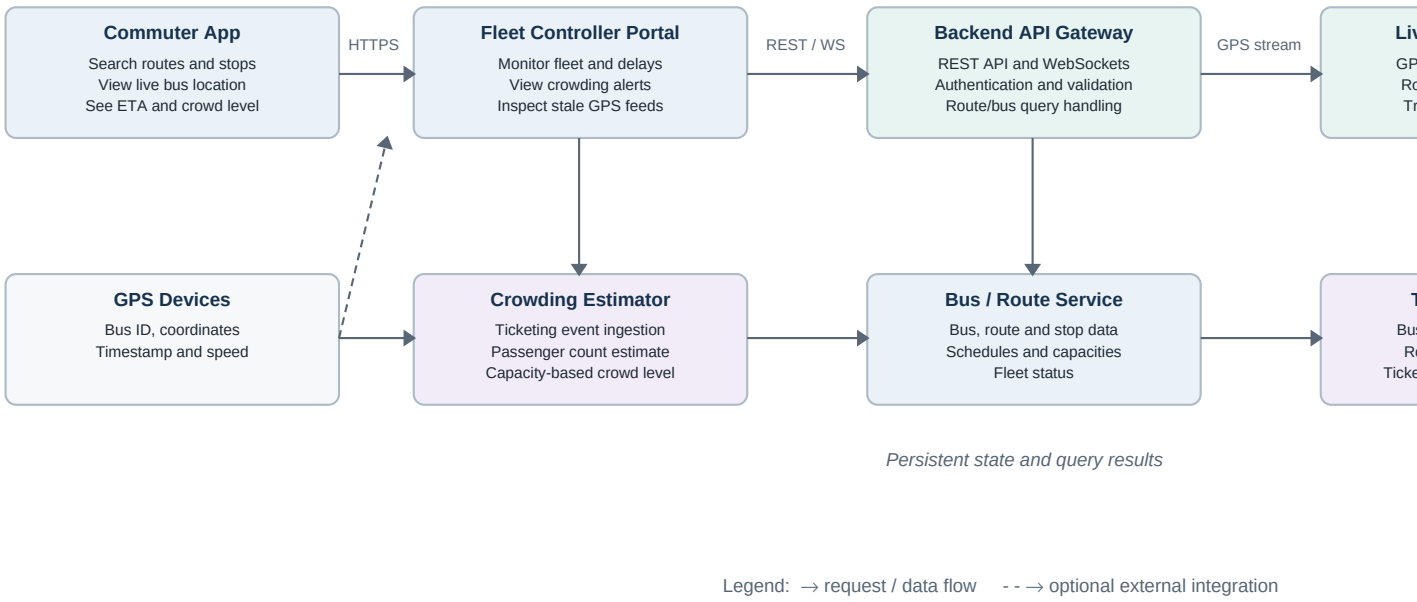
Security Advantage

The API gateway can authenticate users and validate requests, while fleet devices can be authenticated at the telemetry-ingestion boundary. Role-based authorization restricts fleet-management operations to controllers. Service-to-service communication should be protected, and sensitive account data should be kept separate from operational telemetry where practical.

Performance Benefit

The tracking service can prioritize frequent GPS updates and ETA recalculation, while the crowding service processes boarding/alighting events and updates occupancy estimates. The commuter app can receive refreshed results through WebSockets, reducing the need for repeated manual refreshes. The design should be load-tested under simulated peak vehicle and passenger-event volumes.

UML Component Diagram – Microservices Architecture



Legend: → request / data flow - - → optional external integration

Key: Solid arrows show primary request/data flow. Dashed arrows indicate optional external integrations. Traffic data is optional; if unavailable, ETA is calculated from available GPS, route and speed information. Crowding is an estimate derived from ticketing data and may differ from actual occupancy.